

Khant Thu Aung / 105292912

4.1P – ShapeDrawer

Shape.cs

```
using System.Runtime.InteropServices;
```

```
using SplashKitSDK;
```

```
namespace ShapeDrawer
```

```
{
```

```
    public abstract class Shape
```

```
    {
```

```
        private Color _color;
```

```
        private float _x;
```

```
        private float _y;
```

```
        private bool _selected = false;
```

```
        public Shape(Color color)
```

```
        {
```

```
            _color = color;
```

```
            _x = 0.0f;
```

```
            _y = 0.0f;
```

```
        }
```

```
        protected Shape() : this(Color.Yellow)
```

```
        {
```

```
        }
```

```
        public Color Color
```

```

{
    get { return _color; }
    set { _color = value; }
}

public float X
{
    get { return _x; }
    set { _x = value; }
}

public float Y
{
    get { return _y; }
    set { _y = value; }
}

public bool Selected
{
    get {return _selected;}
    set {_selected = value;}
}

//abstract methods
public abstract void Draw();
public virtual bool isAt(Point2D pt)
{
    return false;
}

public abstract void DrawOutline();

```

```
}  
}
```

MyRectangle.cs

```
using SplashKitSDK;  
namespace ShapeDrawer  
{  
    public class MyRectangle : Shape  
    {  
        private int _width;  
        private int _height;  
        /// Constructors  
        public MyRectangle(Color color, int x, int y, int width, int height) : base(color)  
        {  
            X = x;  
            Y = y;  
            Width = width;  
            Height = height;  
        }  
        public MyRectangle() : this(Color.Green, 0, 0, 112, 112) { }  
        /// Properties  
        public int Width  
        {  
            get { return _width; }  
            set { _width = value; }  
        }  
        public int Height  
        {
```

```

        get { return _height; }
        set { _height = value; }
    }

    public override void Draw()
    {
        if (Selected)
        {
            DrawOutline();
        }

        SplashKit.FillRectangle(Color, X, Y, _width, _height);
    }

    public override void DrawOutline()
    {
        int outlineWidth = 7;

        SplashKit.FillRectangle(Color.Black, X - outlineWidth, Y - outlineWidth, _width +
outlineWidth * 2, _height + outlineWidth * 2);
    }

    public override bool isAt(Point2D pt)
    {
        return (pt.X >= X && pt.X <= X + _width && pt.Y >= Y && pt.Y <= Y + _height);
    }
}

```

MyCircle.cs

using SplashKitSDK;

```

namespace ShapeDrawer
{
    public class MyCircle : Shape
    {
        private int _radius;
        public MyCircle() : this(Color.Blue, 62)
        {
        }
        public MyCircle(Color color, int radius) : base(color)
        {
            _radius = radius;
        }
        /// Properties
        public int Radius
        {
            get { return _radius; }
            set { _radius = value; }
        }
        //overridden methods
        public override void Draw()
        {
            if (Selected)
            {
                this.DrawOutline();
            }
            SplashKit.FillCircle(Color, X, Y, _radius);
        }
    }
}

```

```

public override void DrawOutline()
{
    int outlineWidth = 2;
    SplashKit.FillCircle(Color.Black, X, Y, Radius + outlineWidth);
}

public override bool isAt(Point2D pt)
{
    // return (pt.X >= X - Radius && pt.X <= X + Radius && pt.Y >= Y - Radius &&
    pt.Y <= Y + Radius);

    Circle c = SplashKit.CircleAt(X, Y, Radius);
    return SplashKit.PointInCircle(pt, c);
}
}

```

MyLine.cs

```

using System.Runtime.Intrinsics.X86;
using SplashKitSDK;
namespace ShapeDrawer
{
    public class MyLine : Shape
    {
        private float _endX;
        private float _endY;
        public MyLine() : this(Color.Red)
        {
        }

        public MyLine(Color color):base(color)

```

```

{
    X = SplashKit.MouseX();
    Y = SplashKit.MouseY();
    _endX = X + 200;
    _endY = Y;
}

//properties
public float EndX
{
    get {return _endX;}
    set {_endX = value;}
}

public float EndY
{
    get {return _endY;}
    set {_endY = value;}
}

//overridden methods
public override void Draw()
{
    if (_selected)
    {
        DrawOutline();
    }

    SplashKit.DrawLine(Color, X, Y, _endX, _endY);
}

public override void DrawOutline()

```

```

{
    int radius = 3;

    SplashKit.DrawCircle(Color.Black, X, Y, radius); //start circle

    SplashKit.DrawCircle(Color.Black, _endX, _endY, radius); //end circle
}

public override bool isAt(Point2D pt)
{
    float minX = Math.Min(_x, _endX) - 3;
    float maxX = Math.Max(_x, _endX) + 3;
    float minY = Math.Min(_y, _endY) - 3;
    float maxY = Math.Max(_y, _endY) + 3;

    return (pt.X >= minX && pt.X <= maxX && pt.Y >= minY && pt.Y <= maxY);
}
}
}

```

Program.cs

```

using System;
using SplashKitSDK;

namespace ShapeDrawer
{
    public class Program
    {
        private enum ShapeKind
        {
            Rectangle,

```



```

        Circle,
    }

    public static void Main()
    {
        Window window = new Window("Shape Drawer", 800, 600);
        Drawing myDrawing = new Drawing();
        ShapeKind kindToAdd = ShapeKind.Circle;
        do
        {
            SplashKit.ProcessEvents();
            SplashKit.ClearScreen();

            if(SplashKit.KeyTyped(KeyCode.RKey))
            {
                kindToAdd = ShapeKind.Rectangle;
            }
            if(SplashKit.KeyTyped(KeyCode.CKey))
            {
                kindToAdd = ShapeKind.Circle;
            }

            if (SplashKit.MouseClicked(MouseButton.LeftButton))
            {
                Shape newShape;
                switch (kindToAdd)
                {
                    case ShapeKind.Circle:

```

```

        newShape = new MyCircle();
        break;
    default:
        newShape = new MyRectangle();
        break;
    }
    newShape.X = SplashKit.MouseX();
    newShape.Y = SplashKit.MouseY();
    myDrawing.AddShape(newShape);
}

if (SplashKit.KeyTyped(KeyCode.SpaceKey))
{
    myDrawing.Background = SplashKit.RandomColor();
}

if (SplashKit.MouseClicked(MouseButton.RightButton))
{
    myDrawing.SelectShapesAt(SplashKit.MousePosition());
}

if (SplashKit.KeyTyped(KeyCode.DeleteKey) ||
    SplashKit.KeyTyped(KeyCode.BackspaceKey))
{
    foreach (Shape s in myDrawing.SelectedShapes)
    {
        myDrawing.RemoveShape(s);
    }
}

```

```

if (SplashKit.KeyTyped(KeyCode.LKey))
{
    int count = 2; //studentID= 105292912

    for (int i = 0; i < count; i++)
    {
        float startX = SplashKit.MouseX();
        float startY = SplashKit.MouseY() + i * 40;

        MyLine newLine = new MyLine();
        newLine.X = startX;
        newLine.Y = startY;
        newLine.EndX = startX + 200;
        newLine.EndY = startY; // horizontal line

        myDrawing.AddShape(newLine);
    }

    myDrawing.Draw();
    SplashKit.RefreshScreen();
} while (!window.CloseRequested);

}

}

```

